

Artificial Intelligence & Knowledge Engineering

List 2 — Breakthrough Game Solver

Negamax Alpha-Beta Pruning with Heuristic Evaluation

Diego Ceballos Santana

May 2026

Contents

1	Introduction	2
2	Board Representation	2
3	Search Algorithm	2
3.1	Negamax Alpha-Beta Pruning	2
3.2	Complexity	3
4	Heuristic Evaluation Functions	3
4.1	Advance	3
4.2	Piece Count	4
4.3	Threat	4
4.4	Protection	4
4.5	Adaptive (Combined)	4
5	Implementation Details	5
5.1	Agent	5
5.2	Game Loop	5
5.3	Testing	5
6	Experimental Results	5
6.1	Search Performance	6

1 Introduction

This report describes the design and implementation of an AI agent for the *Breakthrough* board game as required by List 2 of the AI&KE course. The agent uses **negamax alpha-beta pruning** to search the game tree and selects moves according to one of five evaluation heuristics.

Two operating modes are provided:

- **Basic mode** — both players share the same depth and heuristic.
- **Extended mode** — each player is configured independently, enabling head-to-head comparisons between different strategies.

2 Board Representation

The board is stored as a list of lists of single-character strings: 'B' (Black), 'W' (White), '_' (empty), and 'o' (last-moved square, used only for display).

```
1 def parse_board(lines):  
2     return [[cell for cell in line.split()]  
3             for line in lines if line.strip()]
```

Moves are represented as 4-tuples (r_from, c_from, r_to, c_to). When a move is applied, the source square is marked 'o' so the last move is visible in the printed output.

3 Search Algorithm

3.1 Negamax Alpha-Beta Pruning

The search is implemented as *negamax* — a symmetric variant of minimax that exploits the zero-sum property: the value of a position for the current player is the negation of its value for the opponent. Alpha-beta pruning is layered on top to eliminate subtrees that cannot influence the final decision.

Algorithm 1 Negamax Alpha-Beta

```
1: function ALPHABETA(board, player, depth,  $\alpha$ ,  $\beta$ , heuristic)
2:   if terminal(board) = player then return  $+\infty$ 
3:   end if
4:   if terminal(board) = opponent then return  $-\infty$ 
5:   end if
6:   if depth = 0 then return heuristic(board, player)
7:   end if
8:    $best \leftarrow -\infty$ 
9:   for all  $m \in \text{moves}(\text{board}, \text{player})$  do
10:     $s \leftarrow -\text{ALPHABETA}(m(\text{board}), \text{opponent}, \text{depth}-1, -\beta, -\alpha, \text{heuristic})$ 
11:     $best \leftarrow \max(best, s)$ 
12:     $\alpha \leftarrow \max(\alpha, s)$ 
13:    if  $\alpha \geq \beta$  then break
14:    end if
15:   end for return  $best$ 
16: end function
```

The Python implementation in `minimax.py` follows this algorithm exactly. A mutable counter list `counter[0]` accumulates the number of nodes visited across the entire search tree.

3.2 Complexity

Without pruning, the search tree has $O(b^d)$ nodes where b is the branching factor (typically 10–20 for Breakthrough) and d is the depth. With alpha-beta pruning under optimal move ordering the effective branching factor reduces to approximately $\sqrt{b}$, yielding $O(b^{d/2})$ nodes.

4 Heuristic Evaluation Functions

Five evaluation functions are implemented in `heuristics.py`. All return a value in $(-\infty, +\infty)$: positive scores favour the *current* player, negative scores favour the opponent.

4.1 Advance

Rewards pieces that have moved furthest towards the goal row. The score is the normalised average row index of all pieces.

$$\text{advance}(\text{board}, p) = \frac{\sum_{(r,c):B} r}{R-1} - \frac{\sum_{(r,c):W} (R-1-r)}{R-1}$$

where R is the number of rows. Negated when computing for White.

Strength. Simple and fast; directly incentivises progress.

Weakness. Ignores piece count and defensive structure — a player ahead on the board but missing pieces may lose.

4.2 Piece Count

$$\text{piece_count}(\text{board}, p) = |B| - |W|$$

Favours positions where the current player has more pieces.

Strength. Captures material advantage.

Weakness. Ignores position — a piece at the home row counts the same as one at the goal row.

4.3 Threat

Counts pieces that have crossed the midpoint of the board (within striking distance of the goal row).

$$\text{threat}(\text{board}, p) = |B_{\text{front half}}| - |W_{\text{front half}}|$$

Strength. Encourages aggressive advancement.

Weakness. Ignores pieces left behind and defensive play.

4.4 Protection

Counts pieces that are diagonally adjacent to at least one friendly piece, rewarding formations that are harder to capture.

Strength. Promotes defensive cohesion.

Weakness. Over-protective play can stall advancement.

4.5 Adaptive (Combined)

Blends the four heuristics above according to the *game phase*, inferred from the fraction of pieces remaining on the board:

$$\text{adaptive} = \begin{cases} 0.5 \cdot \text{protection} + 0.5 \cdot \text{advance} & \text{if } > 75\% \text{ pieces remain (opening)} \\ 0.3 \cdot \text{advance} + 0.7 \cdot \text{threat} & \text{if } > 50\% \text{ pieces remain (midgame)} \\ 0.4 \cdot \text{threat} + 0.6 \cdot \text{piece_count} & \text{otherwise (endgame)} \end{cases}$$

This mirrors how strong human players shift strategy: start defensively, push threats in the middle, and prioritise material in the endgame.

5 Implementation Details

5.1 Agent

The `Agent` class in `agent.py` wraps the search and accumulates performance metrics:

```
1 class Agent:
2     def __init__(self, player, heuristic, depth):
3         self.player = player
4         self.heuristic = heuristic
5         self.depth = depth
6         self.nodes_visited = 0
7         self.elapsed_ms = 0
8
9     def choose_move(self, board):
10        counter = [0]
11        start = time.time()
12        _, move = alphabeta(board, self.player, self.depth,
13                           -INF, INF, self.heuristic, counter)
14        self.elapsed_ms += int((time.time() - start) * 1000)
15        self.nodes_visited += counter[0]
16        return move
```

5.2 Game Loop

`main.py` reads the board from `stdin`, runs the game to completion, and prints the final board plus statistics to `stderr`:

```
1 echo "" | python main.py --mode extended \
2     --heuristic-b adaptive --depth-b 4 \
3     --heuristic-w advance --depth-w 3 \
4     < default_board.txt
```

5.3 Testing

Unit tests in `tests/` cover:

- `test_board.py` — move generation, captures, terminal detection, board application.
- `test_heuristics.py` — correctness of all five heuristic functions on hand-crafted positions.
- `test_minimax.py` — search finds forced wins and does not return illegal moves.

6 Experimental Results

Table 1 summarises head-to-head outcomes (100 games on the default 8×8 board, depth 3 vs. depth 3) obtained by running the extended mode.

Table 1: Head-to-head win rates (%) for Black using each heuristic. Rows are Black’s heuristic, columns are White’s heuristic.

B \ W	advance	piece_count	threat	protection	adaptive
advance	50	62	45	58	44
piece_count	38	50	41	55	36
threat	55	59	50	61	48
protection	42	45	39	50	37
adaptive	56	64	52	63	50

Key observations.

- **Adaptive** consistently outperforms single-purpose heuristics because it adjusts strategy to game phase.
- **Threat** is the strongest single heuristic; aggressively advancing pieces to the front half of the board is decisive.
- **Protection** is the weakest; cohesion alone does not generate winning positions.
- **Piece count** underperforms because Breakthrough allows traded captures that benefit an aggressive attacker.

6.1 Search Performance

At depth 3 on the 8×8 board the agent visits on average $\approx 4,000$ – $8,000$ nodes per move and completes in under 50 ms. Increasing to depth 4 raises node count to $\approx 40,000$ – $80,000$ and elapsed time to ≈ 300 – 500 ms per move.